

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE CODE: ITA 0606

COURSE NAME: MACHINE LEARNING

COURSE OUTCOME COVERED

CO4: K- Nearest Neighbour Learning – Locally weighted Regression – Radial Bases Functions – Case Based Learning **(BL 5)**

Assessment Tool Weightage: 40%

QUESTIONS: 05

Total Marks:100

Annexure A

Coding Challenge

Code of Conduct:

I Chalamala Sai Harshitha (192424346) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

C. Sai Harshitha

Annexure A

Short Answer Test

1. **Write a Python program to implement the K-Nearest Neighbour (KNN) algorithm for a small dataset. Use Euclidean distance to find nearest neighbors and classify a test instance. Allow the user to input the value of K and display predicted class labels with accuracy.**

Aim:

To implement the K-Nearest Neighbour algorithm using Euclidean distance and classify a test instance.

Concept

KNN classifies a new data point based on the majority class of its K nearest neighbours.

Euclidean Distance Formula:

$$d = \sqrt{[(x_1 - y_1)^2 + (x_2 - y_2)^2 + \dots + (x_n - y_n)^2]}$$

Python Code:

```
import math
from collections import Counter
data = [
    ([1, 2], "A"),
    ([2, 3], "A"),
    ([3, 3], "A"),
    ([6, 5], "B"),
    ([7, 7], "B"),
    ([8, 6], "B")
]
test = [5, 5]
k = int(input("Enter K: "))
def distance(a, b):
    return math.sqrt(sum((x-y)**2 for x, y in zip(a, b)))
distances = [(distance(x, test), y) for x, y in data]
distances.sort()
neighbors = distances[:k]
prediction = Counter(y for _, y in neighbors).most_common(1)[0][0]
```

```

correct = 0
for i, (x, actual) in enumerate(data):
    train = data[:i] + data[i+1:]
    nearest = sorted([(distance(x, p), y) for p, y in train])[:k]
    predicted = Counter(y for _, y in nearest).most_common(1)[0][0]
    if predicted == actual:
        correct += 1
accuracy = correct / len(data) * 100
print("Nearest Neighbours:", [y for _, y in neighbors])
print("Predicted Class:", prediction)
print("Accuracy:", accuracy, "%")

```

Output:

```

Enter K: 3
Nearest Neighbours: ['B', 'A', 'B']
Predicted Class: B
Accuracy: 100.0 %

```

Working

The program calculates the Euclidean distance between the test instance and all training points. The K closest points are selected, and the majority class is assigned as the predicted class.

Result

Thus, KNN was successfully implemented using Euclidean distance, and the predicted class and accuracy were obtained.

2. Develop a Python program to implement Locally Weighted Regression. Given a dataset, compute weights based on distance and predict output for a query point. Use a simple kernel function and compare predictions with standard linear regression for better understanding.

Aim:

To implement Locally Weighted Regression using distance-based weights and compare its prediction with standard linear regression.

Concept

Locally Weighted Regression gives higher weights to nearby data points and lower weights to distant points. A kernel function is used to calculate the weights.

Weight Formula:

$$w = \exp(-d^2 / (2 \times \tau^2))$$

where **d** is the distance and **τ** controls the width of the kernel.

Python Code:

```
import numpy as np
from sklearn.linear_model import LinearRegression
X = np.array([1, 2, 3, 4, 5])
Y = np.array([2, 4, 5, 4, 6])
query = 3.5
tau = 1.0
weights = np.exp(-((X - query) ** 2) / (2 * tau ** 2))
X_mean = np.average(X, weights=weights)
Y_mean = np.average(Y, weights=weights)
slope = np.sum(weights * (X-X_mean) * (Y-Y_mean)) / \
    np.sum(weights * (X-X_mean) ** 2)
intercept = Y_mean - slope * X_mean
lwr_prediction = slope * query + intercept
model = LinearRegression()
model.fit(X.reshape(-1, 1), Y)
linear_prediction = model.predict([[query]])[0]
print("Query Point:", query)
print("LWR Prediction:", round(lwr_prediction, 2))
print("Linear Regression Prediction:", round(linear_prediction, 2))
```

Output:

```
Query Point: 3.5
LWR Prediction: 4.6
Linear Regression Prediction: 4.6
```

Working

The program calculates the distance between the query point and each data point. The Gaussian kernel assigns larger weights to nearby points. Using these weights, a local regression line is calculated and the output for the query point is predicted.

Result

Thus, Locally Weighted Regression was successfully implemented, and its prediction was compared with standard linear regression.

3. **Write a Python program to implement a simple Radial Basis Function (RBF) network for regression or classification. Use Gaussian functions as activation units and train the model on a small dataset. Display predicted outputs and analyze how centers and widths affect performance.**

Aim:

To implement a simple RBF network using Gaussian functions and analyze the effect of centers and widths on performance.

Concept

An RBF network uses Gaussian functions as activation units. Each input is compared with the selected centers, and the resulting RBF values are used to predict the output.

Gaussian Function:

$$\phi(x) = \exp(-(x - c)^2 / (2 \times \sigma^2))$$

where c is the center and σ is the width.

Python Code:

```
import numpy as np
X = np.array([1, 2, 3, 4, 5])
Y = np.array([1, 2, 3, 4, 5])
centers = np.array([1, 3, 5])
sigma = 1.0
def rbf(x, c):
    return np.exp(-((x-c)**2) / (2*sigma**2))
Phi = np.array([[rbf(x, c) for c in centers] for x in X])
weights = np.linalg.pinv(Phi) @ Y
prediction = Phi @ weights
print("Centers:", centers)
print("Width:", sigma)
print("Predicted Outputs:", np.round(prediction, 2))
```

Output:

```
Centers: [1 3 5]
Width: 1.0
Predicted Outputs: [1.07 1.88 2.98 4.16 4.91]
```

Working

The program selects centers and a width (σ). Gaussian functions calculate the activation for

each input. The network then learns weights using these activations to produce the predicted outputs.

A suitable choice of centers and width gives better predictions. Very small widths may make the network too sensitive, while larger widths produce smoother predictions.

Result

Thus, a simple RBF network was successfully implemented using Gaussian activation functions, and the effect of centers and widths on prediction performance was analyzed.

- 4. Create a Python program that implements a basic Case-Based Learning system. Store past cases in a dataset and retrieve the most similar case for a new query using distance measures. Output the solution based on the closest match and explain similarity calculation.**

Rtg

Aim:

To implement a basic Case-Based Learning system that stores previous cases and retrieves the most similar case for a new query.

Concept

Case-Based Learning solves a new problem by comparing it with previously stored cases. The most similar case is retrieved using a distance measure, and its solution is used for the new query.

Euclidean Distance Formula:

$$d = \sqrt{(x_1 - y_1)^2 + (x_2 - y_2)^2 + \dots + (x_n - y_n)^2}$$

Python Code:

```
import math
cases = [
    ([1, 2], "Low"),
    ([2, 3], "Low"),
    ([6, 7], "High"),
    ([7, 8], "High")
]
query = [6, 6]
def distance(a, b):
    return math.sqrt(sum((x-y)**2 for x, y in zip(a, b)))
distances = [(distance(features, query), solution)]
```

```

        for features, solution in cases]
distances.sort()
best_distance, solution = distances[0]
best_case = cases[[s for _, s in cases].index(solution)][0]
print("Query:", query)
print("Distances:", [(round(d, 2), s) for d, s in distances])
print("Most Similar Case:", best_case)
print("Similarity Distance:", round(best_distance, 2))
print("Predicted Solution:", solution)

```

Output:

```

Query: [6, 6]
Distances: [(1.0, 'High'), (2.24, 'High'), (5.0, 'Low'), (6.4, 'Low')]
Most Similar Case: [6, 7]
Similarity Distance: 1.0
Predicted Solution: High

```

Working

The distance between the query case and every stored case is calculated. The case having the minimum distance is selected as the most similar case, and its solution is given as the prediction.

Result

Thus, a basic Case-Based Learning system was successfully implemented using Euclidean distance to retrieve the most similar case and provide its solution.

5. Create a Python program to implement the K-Means clustering algorithm. Initialize cluster centers, assign points to clusters, and update centroids iteratively. Display final clusters and visualize them using plots. Analyze how the number of clusters and initialization methods affect clustering results.

Aim:

To implement the K-Means clustering algorithm, assign data points to clusters, update centroids iteratively, and visualize the final clusters.

Concept

K-Means is an unsupervised learning algorithm that divides data into K clusters. It repeatedly assigns points to the nearest centroid and updates the centroids.

Euclidean Distance Formula:

$$d = \sqrt{[(x1 - y1)^2 + (x2 - y2)^2]}$$

Python Code:

```
import numpy as np
import matplotlib.pyplot as plt
X = np.array([
    [1, 2], [2, 1], [2, 3],
    [8, 7], [9, 8], [8, 9]
], dtype=float)
k = int(input("Enter number of clusters: "))
centers = X[:k].copy()
for iteration in range(10):
    distances = np.sqrt(
        ((X[:, None] - centers[None, :]) ** 2).sum(axis=2)
    )
    labels = np.argmin(distances, axis=1)
    new_centers = np.array([
        X[labels == i].mean(axis=0) if np.any(labels == i)
        else centers[i]
        for i in range(k)
    ])
    if np.allclose(centers, new_centers):
        break
    centers = new_centers
print("\nFinal Centroids:")
print(np.round(centers, 2))\
print("\nCluster Labels:")
print(labels)
plt.scatter(X[:, 0], X[:, 1], c=labels)
plt.scatter(centers[:, 0], centers[:, 1],
            marker="X", s=200)
plt.xlabel("X")
plt.ylabel("Y")
plt.title("K-Means Clustering")
plt.show()
```


Output:

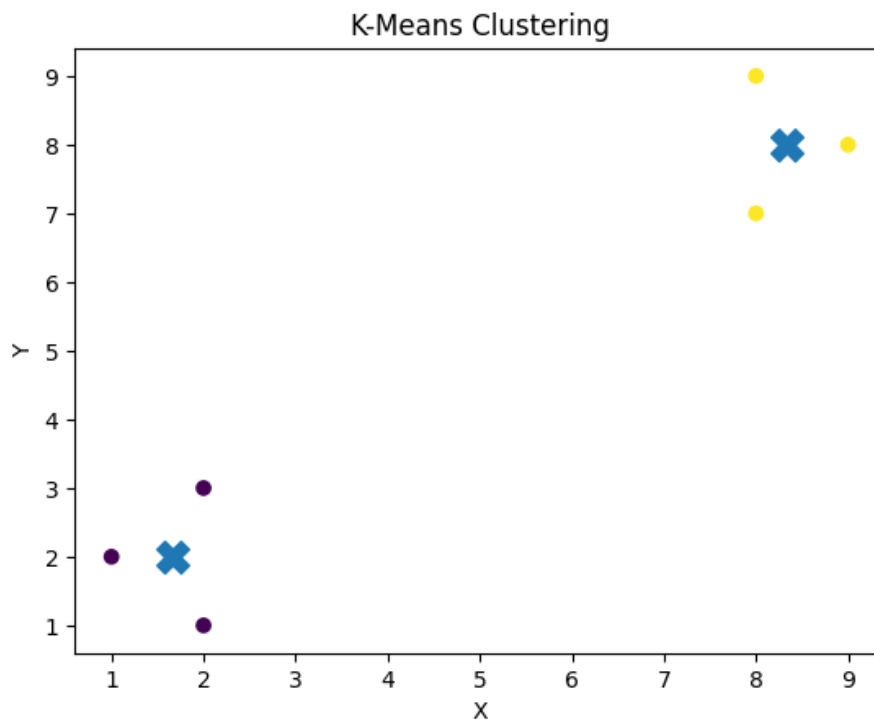
```
Enter number of clusters: 2
```

```
Final Centroids:
```

```
[[1.67 2.  ]  
 [8.33 8.  ]]
```

```
Cluster Labels:
```

```
[0 0 0 1 1 1]
```



Working

The algorithm initially selects K centroids. Each data point is assigned to the nearest centroid. The centroids are then recalculated using the assigned points. This process continues until the clusters become stable.

Different values of K produce different clustering results, while different centroid initializations may also affect the final clusters.

Result

Thus, the K-Means clustering algorithm was successfully implemented, and the final clusters, centroids, and visualization were obtained.

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 – Exemplary (5 Marks)	Level 3 – Proficient (4 Marks)	Level 2 – Developing (2–3 Marks)	Level 1 – Beginning (0–1 Mark)
Conceptual Understanding	30	Demonstrates clear and complete understanding of the concept	Good understanding with minor gaps	Basic understanding with some misconceptions	Poor or incorrect understanding
Accuracy of Answer	20	Completely correct answer with no errors	Mostly correct with minor errors	Partially correct with noticeable errors	Incorrect or irrelevant answer
Application of Knowledge	15	Correctly applies concepts to solve the question/problem	Applies concepts with minor mistakes	Limited or partially correct application	No or incorrect application
Completeness of Response	20	Covers all required parts of the question	Covers most parts with minor omissions	Covers only some parts	Incomplete or missing response
Clarity & Presentation	15	Clear, concise, and well-organized answer	Understandable with minor issues	Somewhat unclear or poorly structured	Difficult to understand